

what
is
slop
ware

,
how
do
you
spot
it,
what
do
you
do?

a
curricul
um that
teaches
you to
measure
the
distance
between
somethin
g that
looks
like it
works
and
somethin
g that
actually
holds

up.
every
chapter
ends
with a
check
you run
on your
own
project.

6 / 55
chapters
· pdf ·
every
published
chapter
on one
page. for
printing
or the
pdf.

00 why
does
this
book
exist?
01 what
is the
thing
on your
screen?
02 why
can
your
deploy
not
find

your
files?

03 why
does
your
link
only
open
for
you?

04 what is
the
difference
between
npm run
dev and
npm run
build?

05 why
does the
till
stand in
a
different
room?

why does this book exist ?

Why this
book
exists,
who is
writing
it, and
how to
read it.

I have
been using
forums
such as
Reddit,
Quora and
Ekşi
Sözlük for
years, and
once I was
one of the
people
posting
there.
People
make fun
of
localhost:3
000 users,
and they
believe CS
is dead
because

somebody
developed
a local
website.
Actually, I
hate
needless
jokes
made just
to fit in.
But also,
lack of
technical
depth gave
those
localhost:3
000
owners the
courage of
a
superhero.
So instead
of joining
in, I
decided to
create a
blog about
how to
avoid
slopware,
based on
my own
experience
s.

Learnin
g it cost
me 53
repos, 8
pull
requests
and 6
Claude
Max x20
accounts.
It also
cost me

exploding
token bills,
sleepless
nights,
and the 5
kilos I lost
to hunger
and lack
of sleep. I
do not
want that
to happen
to
anybody
else, and I
do not
want a
power this
large to
stay
hidden
inside one
class of
people. I
am using
Claude
while I
write this,
and I am
sure Dario
Amodei
would
want the
same. So I
am writing
it with my
own
hands, out
of
everything
I pulled
from my
own pain.
I do
not believe
that CS

has been
replaced
by AI.*
Over time
there was
COBOL
and C++
did not
replace it,
Java did
not
replace
C++, and
Python
did not
replace
Java.
Given the
nature of
high level
languages,
I treat an
LLM as
one more
high level
language.
(And
where has
engineerin
g gone?
Engineerin
g is the
difference
in the
thinking
itself, in
knowing
what to
make and
in
connecting
technologi
es and
developing
them.)

I love
vibecodin
g

Tony
Stark is a
vibecoder
with
unlimited
tokens. As
far as I
see,
internships
and
projects
are
presented
as though
they
belong to
certain
people.
What gets
inherited
is not only
money.
You can
call it
inherited
vision, you
can call it
social
climbing,
and there
is even a
niche for it
now,
something
like a class
based
networkin
g hub
founder.
What will
happen
soon is

that
somebody
starts a
business
with a
Claude
Pro
account
and has
graduates
of good
schools
working
underneat
h them,
when
those same
students
could have
started
that
company
with their
pocket
money.
Neverthele
ss, if it is
not rocket
science
and it is
not tank
design, I
do not
think any
work that
needs no
large
capital is
as big and
as
irreplaceab
le as it is
claimed to
be. Your
applicatio
n was

rejected,
so say that
you could
do this too
and carry
on your
way.

Everyo
ne should
benefit
from
technology
, and not
only for
business
and
money,
but for
making
products
for your
own needs.

As I
believe
from
Plato, no
one does
wrong
willingly,
and
nobody
writes bad
software
on
purpose
either.

Outside
academic
purposes,
CS is no
different
from being
able to
sew a
button on.
Everyone

should
learn it in
order to
make the
ideas they
dream of
real, and
most
importantl
y,
everyone
can,
because I
believe
that for
once in
history
anyone
can build
anything.

What
matters is
whether
anything
stands
behind the
code.
Think
about the
last time
your
terminal
told you
every test
passed. It
told you
that some
tests ran
and none
of them
complaine
d, and
nothing
about
whether
they

would
have
noticed a
broken
app.

Who am
I?

I am
Damla, a
CS
student at
Bilkent. I
was on the
board of
IEEE, I
worked as
a Python
assistant, I
did an
internship
and a
fellowship,
and the
rest is on
my CV.

Actual-
ly I was a
coder
before CS
took over,
starting
with
HTML at
11 and a
lot of
Stack
Overflow.
Now I
vibecode
anyway.
(Okay,
what I

really do
is LLM
systems
engineerin
g, which
means I
connect
systems
and
engineer
them.)

ir-globe
draws
what 198
countries
do to each
other and
it has
been going
for months
without
me
touching
it, on
infrastruct
ure that
costs 0
dollars a
month.
stitchu
takes a
photograp
h of a
garment
and gives
back a
sewing
pattern,
and it has
a gate
that
decides
whether
that
pattern
can

actually
be sewn,
which I
wrote
after
printing
patterns
that
passed
every test
I had and
could not
be sewn at
all.
lulumelon
measures
what
language
models say
about a
brand and
prints a
range
rather
than a
single
number,
because on
its first
live run
the honest
interval
ran from 0
to 33,3.
rabadon is
a gate
system for
agents,
written in
C++ with
no
dependenc
ies, and it
exists
because an
agent once

told me it
had
finished a
job it had
not
finished.

Each of
those
started as
something
I got
wrong,
then
found,
then fixed,
and the fix
is what
this book
hands
over.

How
should
you read
this
book?

The order
is
semantic,
and I
spent a
full week
reading
and
thinking
before I
settled it.
I started
from
localhost,
because
that is the
joke I kept

seeing.

Writing it
made me
realise the
link only
fails if you
believe a
page is a
place, so
that
subject
had to
come first.

Writing
that one
made me
realise I
was
talking
about files
without
ever
saying
what a
folder is,
so the
terminal
came next.

Localho
st then
required
npm run
dev,
because
once you
know
whose
machine
answers
you want
to know
what is
running on
it. That
one
required

npm run
build,
because
the thing
you would
put on
another
machine is
not the
thing
running on
yours.
After that
came the
backend,
because a
program
on another
machine is
what the
whole
book has
been
walking
towards.
The
backend
required
data,
because a
backend
keeps
something.
Data
required
.env files
and API
keys,
because
the
moment
something
is kept,
whatever
reaches it
matters.

Those
chapters
are
grouped
into five
parts. 101
is what
you are
actually
holding.
201 gets
your
project off
your own
computer.
301 is
where real
users and
real
attackers
arrive, 401
is the part
people
touch, and
501 is
money.
Afterward
s I will
write
about
LLM
system
design,
orchestrati
on, loops,
workflows
and gates.

Every
chapter
opens with
a real
incident
and closes
with the
thing you
can do

tomorrow,
on your
own
project. I
don't want
to stretch
the
schedule
because 2
months in
tech = 10
years of
developme
nt in any
topic.

By the
way, new
tabs keep
opening in
my head,
and a
chapter
about one
thing
wakes up
an idea
about
something
else
entirely.

Those
ideas do
not belong
inside the
chapter, so
they go in
the
appendix,
and terms
and small
technical
things
that do
not fit
there go in
the

footnotes.
Both sit at
the
bottom of
the page,
so
anything
marked
with a star
has a note
waiting for
it.

The
appendix
is also
where the
writing
about
product
manageme
nt and the
startup
world will
go.
Companies
still run
events to
hand out
the kind of
knowledge
you can
only get
through
experience
, and then
they turn
creative
people
away with
reasons on
the level
of you are
a Gemini,
that is
why you
were

eliminated
. To tell
you the
truth, I do
not think
anybody is
as well
intentione
d as I am,
and people
hold a
strange
principle
about not
sharing
good
things.
You can
steal my
ideas.
There is
sand in
the sea
and there
are ideas
in me, so
passing on
what I
know is
nothing
but a
happiness
for me.

Thanks

Years ago
the yazbel
documenta
tion made
me
interested
in
computer
science,

and I
changed
my
departmen
t. Even
though we
have not
been
writing
code by
hand for
the last
year, it
gave me a
love of CS
that no
undergrad
uate
degree
could have
given me.
We used
to shorten
loops with
algorithm
analysis,
and now
we do the
same thing
inside
workflows.
The
language
changed
and the
tool
changed,
while the
purpose
and the
logic did
not, so I
still think
this
motion in
CS is a

matter of
perspective.
e.

So
when I am
no longer
a sweet
student
but a
businesswo
man, I
hope this
book will
have
enlightene
d
somebody,
entertaine
d
somebody,
or made
somebody
think.

Two
heads are
better
than one,
so
whatever
work you
are in, try
your own
idea too.
My
endless
respect to
everyone
who is
dreaming
and
working
for
something.

* Is CS
dead?
The
claim
is
usuall
y
made
about
a
langu
age,
not
about
the
field.
COB
OL
was
going
to be
replac
ed by
C++,
C++
by
Java,
Java
by
Pytho
n,
and
every
one of
them
is still
runni
ng

some
where
right
now.
What
chang
ed
each
time
was
the
height
of the
langu
age,
not
wheth
er
some
body
had
to
decide
what
to
build.

what
is
the
thin
g on
your
scree
n?

A page
is not a
place.
It is a
delivery
that
happens
again
every
time
somebody
looks.

The first
thing you
do with a
coding
agent is
ask it for a
website,
and a
minute
later there
is one.
The
terminal
prints an
address.
You open

it and the
thing you
described
is sitting
in the
browser
looking
finished.

Ask
where that
thing lives
and the
answer
comes
back as in
the
browser or
on my
computer
or on the
internet.

All three
feel fine
right up
until
something
breaks,
and then
none of
them tell
you where
to look.¹

¹
**A book
note.** A
model takes
the easiest
road. It
gave you
localhost
with no
backend,
because
that is the
shortest

way to
something
that runs.

What
arrives
when a
page
opens?

Make a
folder
anywhere
you like
and put a
single file
in it called
index.html
with one
line inside.



Open a
terminal
in that
folder and
run
python3 -
m
http.server
8000. Go
to
localhost:8
000 in
your
browser.
The word
hello is
there in
large
letters,
arriving

from a
server
small
enough
that
nothing
can hide
inside it.

Now
right click
on the
page and
choose
view
source.

What
comes up
is the line
you typed,
character
for
character.

Change
the h1 to
a p and
refresh.

The same
word
comes
back
small.

Nobody
sent you a
smaller
word. You
changed
one
instruction
and the
browser
drew the
letters
differently,
because
what
travelled

to it was
never a
page. It
was a set
of
instruction
s for a
program
that
carries
them out.

How does
the
browser
learn
about the
second
file?

One file is
not how
anything
real
arrives.
Next to
index.html
make a file
called
style.css
containing
h1 { color:
crimson; }
and
mention it
from the
page.



Refresh
and the
word is
red.

Your
browser
had no
way of
knowing
that
style.css
existed.

The only
file it had
been given
was
index.html

. It read
that file
and came
to a line
mentionin

g a
stylesheet.

Only then
did it go
back to
the server
for a
second file.

The page
tells the
browser
what else
to fetch,
one
discovery
at a time.

Open
the
developer
tools with
Cmd and
Option
and I on a
Mac or

F12 on
Windows.
Click the
Network
tab and
refresh
with the
panel
open. Two
rows come
up,
index.html
first and
style.css
second,
and they
can never
come in
the other
order.

A slow
page is
rarely one
slow file.
The
document
arrives
and
mentions
something.
That
something
arrives
and
mentions
two more.
Nothing
further
down the
chain
starts
until the
thing
before it
has been
read.

Where
does the
page go
when the
server
stops?

'The page
is on my
computer,
so it will
still be
there,' you
may be
thinking.
In the
Network
panel
there is a
checkbox
marked
Disable
cache.
Tick it so
the
browser
stops
keeping
copies of
its own.
Now go to
the
terminal
and stop
the server
with Ctrl
and C,
then
refresh the
page.

The
page is
gone and
there is no
error
about

your
HTML,
because
your
HTML
never
arrived.
Nothing
had been
sitting in
the
browser
waiting for
you.

Start
the server
again and
refresh.
Hello
comes
back in
red
exactly as
before,
drawn
from
nothing. A
page is a
delivery
that
happens
again
every time
somebody
looks.

What
does this
look like

in a real
project?

Open the
project
you have
been
working
on and
refresh it
with the
panel
open. The
same list
appears,
only far
longer.
The
document
sits at the
top.
Everythin
g the
document
mentioned
sits under
it, and
under
those sits
everything
they
mentioned
in turn.

At the
bottom of
the panel
there is a
total size.
Every
visitor
downloads
that much
before
they see
anything
at all, out

of their
connection
and their
phone
plan.

That
leaves one
question
for the
rest of the
book.

Some
program
on some
machine
sent those
files. On
the hello
page you
know
which one,
because
you
started it
yourself
and it is
still sitting
in your
terminal.
On your
real
project
you have
probably
never
thought
about it.

CHE
CK

Op
en
the

Ne
tw
ork
pa
nel
on
the
pro
jec
t
yo
u
are
pro
ud
est
of
an
d
ref
res
h
it.

Ho
w
ma
ny
file
s
arr
ive
d?

Ho
w
ma
ny
kil
ob
yte

s
did
the
y
co
me
to?

W
hic
h
file
in
tha
t
list
can
yo
u
not
acc
ou
nt
for
?

Th
ere
is
nea
rly
alw
ays
one

.
Us
ual
ly
a
fon
t
yo

u
sto
pp
ed
usi
ng
mo
nth
s
ago
or
a
libr
ary
tha
t
arr
ive
d
att
ach
ed
to
so
me
thi
ng
els
e.
No
thi
ng
an
no
un
ced
it,
bec
aus
e it

was
s
me
nti
one
d
by
a
file
tha
t
was
s
itse
lf
me
nti
one
d
by
a
file
.

No
thi
ng
nee
ds
re
mo
vin
g
tod
ay.
It
has
bee
n
goi
ng

out
to
eve
ry
visi
tor
sin
ce
the
da
y
it
ap
pe
are
d.

☐ my
project
passes
this
check

You just
started a
server
inside a
folder
without
my having
said what
a folder is,
and that
gap has to
close
before we
can ask
whose
machine
your files
come
from. So

the
terminal
and the
folder
come next.

why can your deploy oy not find your files?

The same
command
run one
level up
serves
nothing.
Most
deploy
failures
start
here.

You ran a
command
inside a
folder a
moment
ago and a
server
appeared,
and I
never said
which
folder it
was
reading or
why that

mattered.
The same
command
run from
one step
higher up
would
have
served
nothing.

A
whole
family of
failures
comes out
of that.
The build
that works
on your
machine
and
returns file
not found
on the
deploy is
one, the
image that
loads
locally and
404s in
production
is another.
In every
one of
them a
program is
standing
somewhere
you did
not
expect,
reading a
path that
means
something

different
from
there.

Where
are you
standing?

Open a
terminal.
On a Mac
it is called
Terminal.
On
Windows,
use the
one your
developme
nt tools
installed
rather
than the
old black
box.

A
command
never runs
in general.
It runs
while you
are
standing
somewhere
and where
you are
standing
changes
what it
does.

A terminal window with a light gray background. The prompt character is not visible. The command 'pwd' is entered on a single line.

Print
working
directory.
It answers
with the
absolute
path of
where you
are
standing
right now.



Plain `ls`
lists files
and
folders
together
with no
way to tell
them
apart, and
`-F` puts a
slash on
the end of
every
folder
name. Do
not try to
work it
out from
the name.
`README`
has no dot
and is a
file, while
a folder
called
`backup.old`
has one
and is still
a folder.



Change directory, into the Desktop that sits inside wherever you already are. Run `pwd` again and the answer has grown by one step. Run `ls -F` and the contents have changed, because the same command means something different here.



Two dots is a real entry that exists inside every folder on the disk and it points at

that
folder's
parent.

That is
the whole
of
navigation
. Three
commands
and one
convention
, and they
work the
same way
on every
Mac and
every
Linux
machine
and inside
the
terminal
your
Windows
tools
installed.

Is the
terminal
a
different
place?

If the
terminal
still feels
like a
second
machine
hiding
underneat
h the one
you use,
open

Finder or Explorer and go to your project folder. Put that window on one side of your screen and the terminal on the other.

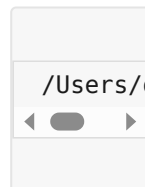
Now `cd` your way to the same folder and run `ls -F`.

They are the same thing. The icons on the left and the names on the right are one folder described in two ways, and the three commands you just learned will reach any folder on the disk.

What is a path?

A folder holds files and other folders and those hold more. All of it hangs down from one starting point, so your whole disk is a single tree.

A path is directions through that tree and pwd printed one of them.



It means start at the top of the disk and go into Users, then into damla and Desktop and project. Each slash is a door you walk through.

A postal address is written the same way and a path is that written on one line.

There are two kinds.



The absolute one begins with a slash and is measured from the top of the disk, so it means the same file wherever you use it.

The relative one is measured from where you are standing, so those same characters point at different files depending

on where
they are
read.

Why is
this so
hard to
learn?

Go back
to the
folder with
index.html
and
style.css in
it and
start the
server
again. The
page loads
and the
word is
red. Now
stop the
server, run
`cd ..` to
step up
one level,
and start
it again
from
there.



Open
localhost:8
000. There
is no page,
only a list
of folder
names.

Nothing
was moved
and
nothing
was
deleted.
The href
inside your
HTML
still says
style.css
and it still
means the
style.css
next to
me, and
the server
is now
standing
one level
up where
no such
file sits.
Your
whole
project is
intact and
unreachabl
e.

That is
the entire
failure,
and here is
why it
stays
invisible
until it
costs you
a day. A
few years
ago
lecturers
in physics
and
engineerin
g started

running
into
something
they could
not
explain.
They
would ask
a class to
open a file
from a
particular
directory.
We as
students
usually
struggle to
do exactly
that,
myself
included,
even
though we
use a
computer
all day
and use it
well.

A file
lives in the
app or on
the
computer,
and the
way you
reach it is
to search
for it.

That
model
works for
daily life.
Put
everything
in one pile
and search

when you
need
something.
You will
find your
holiday
photographs
every
time. It
stops
working at
the deploy,
because
the
machine
you deploy
to has no
search
box. It is
handed a
path and
goes
exactly
there. If
nothing is
there it
stops.

Directory
structure
is so
ordinary
to the
people
who know
it that
words for
it are hard
to find, so
nobody
explains it
and
nobody
asks.

Why does
this
decide
whether
things
work?

Every
build step,
every
deploy
tool and
every error
message
you will
read for
the rest of
your life
assumes
you know
where you
are
standing.

File
does not
exist
almost
always
means a
relative
path was
read from
a folder
you were
not
expecting.

The file
was there
all along,
and the
program
was
standing
somewhere
else when
it went

looking for
it. You
saw
exactly
that a
minute
ago with a
server one
level too
high.

Some-
thing works
on your
machine
and dies
on the
deploy for
the same
reason.

You typed
the
command
while
standing
inside the
project
folder.

The other
machine
runs it
while
standing
wherever
it happens
to start, so
the
relative
path
points at a
place that
does not
exist.

Chapter
16 is

where this
bites
hardest.

MIT
teaches a
free course
for exactly
this gap
called The
Missing
Semester
of Your
CS
Education.
Julia
Evans
draws
comics
about the
shell that
are the
friendliest
thing on
the
subject.

CHE
CK

Op
en
a
ter
mi
nal
an
d
lea
ve
yo
ur
edi
tor

clo
sed
.

Ge
t
to
yo
ur
pro
jec
t
fol
der
usi
ng
onl
y
pw
d,
ls -
F
an
d
cd.
Re
ad
wh
at
co
me
s
bac
k
aft
er
eac
h
ste
p
ins

tea
d
of
typ
ing
the
wh
ole
pat
h
fro
m
me
mo
ry.

W
he
n
yo
u
arr
ive,
wh
ich
fol
der
in
tha
t
list
can
yo
u
not
des
cri
be
the
con
ten

ts
of?
Th
ere
wil
l
be
one
,
usu
all
y
no
de
—
mo
dul
es
or
dis
t
or
.ne
xt.
Go
int
o it
an
d
loo
k,
cha
ngi
ng
not
hin
g.
Th
e
ne

xt
tim
e
so
me
thi
ng
bre
aks
tha
t
fol
der
is
goi
ng
to
ap
pe
ar
in
the
err
or
me
ssa
ge
(w
hic
h
sho
uld
not
be
the
firs
t
tim
e
yo

u
see
the
na
me
).

☐ my
project
passes
this
check

Next
comes
localhost,
and the
reason a
link that
opens
instantly
for you
does not
open for
your
friend.

why
does
your
link
only
open
for
you?

Your
machine
answers
its own
name.

Your
friend's
machine
answers
its own,
and
finds
nothing.

This
chapter is
about
famous
localhost:3
000. Jokes
about it
have been
going
around the
internet
for a long
time.
Many
people

believe CS
is dead
because
they built
a website
hosting on
local. You
click your
own link
and it
opens.
Your
friend
clicks it
and it
does not.
Why does
it happen,
what is
localhost:3
000, and
how to fix
this issue?

What
does
localhost
mean?

It begins
with
localhost:3
000 or
127.0.0.1:3
000, which
are the
same
address
written
two ways.
localhost
is a name
and
127.0.0.1

is the
number
the name
stands for.
RFC 6761
reserves
both for
one
purpose,
and that
purpose is
to mean
the
computer
that is
asking.
When you
type it,
your
machine
sends the
request to
a program
on itself.
The
request
never
leaves the
machine,
which is
why this is
called the
loopback
address.
When
your
friend
types it,
their
machine
sends the
request to
a program
on itself
and finds
none.

Each
computer's
localhost
is a
different
server.

The
number
after the
colon is
the port.
If the
address is
the
building,
the port is
the
apartment
inside it.

One
machine
runs many
programs
and each
waits at
its own
number.

3000 is a
convention
development
tools
picked,
nothing
more. The
port is
fine. The
building is
the
problem,
because
you gave
your
friend the
name
every

building
uses for
itself.

Can your
own
phone
open it?

Open the
terminal
and cd
into any
folder that
has a file
or two in
it. Run
this.

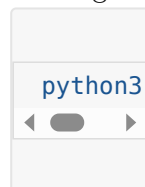


This starts
a small
web
server.

The flag
tells it to
answer
only the
machine it
is running
on. Open
localhost:8
000 in the
browser
and you
see the
folder's
files listed.
Now type
the same
address
into your

phone's
browser.
Nothing
comes
back, even
though the
phone is
on the
same wifi.
The phone
went
looking for
a program
at port
8000 on
itself.

Now
stop the
server
with
Ctrl+C
and start
it again
without
the flag.



The flag
was doing
all the
work.
127.0.0.1
told the
server to
listen on
the
loopback
address
alone, so
requests
arriving
over wifi
were never
answered.

Without
the flag
this tool
listens on
0.0.0.0,
which is
not an
address
you
connect
to. It
means
every
network
interface
the
machine
has,
including
the wifi
card. A
request
from the
phone is
now
accepted
the same
as one
from the
machine
itself. You
will see
0.0.0.0
again in
Docker
logs and in
cloud
deploy
output
and it
always
means the
same
thing.

Now
find the
laptop's
address on
the local
network.
On a Mac
that is
ipconfig
getifaddr
en0 and
on Linux
hostname
-I. It looks
like
192.168.1.
24. If the
Mac
command
comes
back
empty
your wifi
is on
another
interface,
so try en1.
Type
http://192
.168.1.24:8
000 into
the phone
and the
file list
appears.
Nothing in
the files
changed,
only which
doors the
program
answers
and which
machine

the phone
was told
to look at.

How far
does the
local
network
reach?

'So I drop
the flag
and I am
live,' you
may be
thinking.
What you
have is a
server
reachable
from the
same wifi.
192.168.1.
24 is an
address
inside your
home
network
and means
nothing
outside it,
so a friend
in a cafe
or a
customer
on mobile
data
cannot
reach it.
And when
the laptop
lid closes,

nothing is
running
anywhere.

A
product
has to
answer
while you
are asleep,
so the
program
has to run
on a
machine
that stays
on and has
an address
the
internet
can route
to.

Putting it
there is
called
deploying,
and that is
chapter
16. ir-
globe
answers at
three in
the
morning
because it
is not on
my laptop.

CHE
CK

Lo
ok
at
the

ad
dre
ss
bar
of
the
pro
jec
t
yo
u
are
pro
ud
est
of.
If
it
be
gin
s
wit
h
loc
alh
ost
or
127
.0.
0.1
,
the
nu
mb
er
of
pe
opl
e
wh

o
can
op
en
it
is
1.
Th
en
clo
se
the
lap
top
.
Tu
rn
off
wif
i
on
yo
ur
ph
one
so
yo
u
are
on
mo
bil
e
dat
a
an
d
try
to
op

en
the
pro
jec
t.
W
hat
eve
r
co
me
s
bac
k
is
the
ans
we
r
to
wh
eth
er
it
exi
sts
yet
.

☐ my
project
passes
this
check

After that
come the
two
commands
you run

without
reading
them.

Only one
of npm
run dev
and npm
run build
makes a
thing you
can
deploy.

what
is
the
difference
between
npm
run
dev
and
npm
run
build
?

One
builds
pages in
memory
for you.

The
other
writes
the
folder
your
users
get.

Localhost
turned out
to be a
name your
machine
uses for
itself. The
other half
of the
address is
still
unexplaine
d, because
something
is
answering
at that
door, and
in a
framework
project
that
something
is npm run
dev.

You
have run
that
command
a
thousand
times
without
wondering
what it is.
Sitting
next to it
in the
same file
there is
npm run
build. It
takes
much
longer and
produces a

folder you
have
probably
never
opened.
That
folder is
the part
your users
get.

What is
running
when you
run npm
run dev?

What runs
is a
program
sitting in
your
terminal
and
waiting to
be asked
for things,
the same
way
python3 -
m
http.server
did in
chapter 1.
This one is
far
cleverer
about
what it
hands
over.
Go to
the
terminal

where it
says ready
and press
Ctrl and
C. Refresh
the
browser.
The site is
gone and
the error
is a
refused
connection
, which
means
nothing is
listening
at that
door any
more.

Now
start it
again and
open the
project
folder,
looking for
the HTML
that
arrived in
the
browser.
There is
no such
file. Your
project
has a src
folder full
of
component
s and not
one of
them is a
page.
Nothing
on the

disk
matches
what you
saw in
view
source.

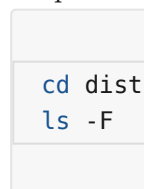
The
dev server
built that
page in
memory
the
moment
your
browser
asked for
it and
then let it
go. Its
whole job
is to read
the source,
work out
what the
browser
needs
right now
and hand
it over.
Then it
watches
the files so
it can do
the same
thing the
instant
you save.
Nothing
was
written
down, so a
save
appears in
the

browser in
under a
second.

What
does the
build do?



It takes a
while,
where the
dev server
started
instantly.
When it
finishes
there is a
new folder
called dist
on Vite
and .next
on Next.js.
Go in
there with
the
commands
from
chapter 2.



The
HTML
that did
not exist a
minute
ago is in
there. So

are the
scripts,
with
names like
index-
4f3a9c.js.
Those
characters
in the
middle are
there so a
browser
can safely
keep an
old copy
without
mistaking
it for the
new one
after you
deploy.
The CSS
from every
component
in the
project
has been
gathered
into one
file.

Your
source is
written in
whatever
shape lets
you work.
This folder
is written
for
browsers,
in one
shape and
once. The
dev server
had been
standing

in for it on
demand
without
ever
writing a
word of it
to disk.

Which
one gets
deployed?

The folder
gets
deployed,
never your
source and
never the
dev server.
Deploying
means
building
that folder
and
putting its
contents
somewhere
that
answers
requests
all day, so
a visitor is
handed a
finished
file instead
of waiting
for a
program
to work
one out.

You
can serve
that folder

yourself
right now.



Open
localhost:8
000. That
is your
site,
coming
out of a
plain file
server that
has never
heard of
your
framework
. It is the
same
server that
handed
you one
line of
HTML in
chapter 1.

Why do
the two
behave
differently?

'Same
code, same
project, so
it will
behave the
same way,'
you may
be
thinking.

Open one
of those
scripts in
dist and
read it.
Your
function
names are
gone,
replaced
by single
letters.

The spaces
are gone.
The whole
file is one
line.

The
dev server
is
optimised
for you. It
starts
instantly
and
refreshes
instantly,
keeps your
variable
names so
errors stay
readable,
and
squashes
nothing.
The build
is
optimised
for the
visitor, so
it makes
the
smallest
files it can,
squashes
everything

together,
drops code
that
nothing
uses and
shortens
every
name it
can reach.

Squashi
ng changes
behaviour.
Code that
quietly
relied on a
function
still
having its
own name
breaks,
because
the name
is gone
and you
just read
the file
where it
went
missing. A
value read
while the
build is
running
gets frozen
into the
file, so
changing
it on the
server
afterwards
changes
nothing
until you
build
again, and
chapter 13

is about
the
trouble
that
causes. A
file the
dev server
found by
walking
your folder
is missing
from the
output
entirely if
nothing
imported
it
properly.

So a
project
can run
beautifully
for weeks
and fall
over on
the first
deploy
without a
line of
source
having
changed.
When
somebody
says it
works on
my
machine
they are
usually
telling the
truth,
because
their
machine
was

running
the
friendly
version.

Before
you
deploy,
run the
build and
serve the
output
locally the
way you
just did,
then click
around
your own
site.

Anything
broken in
that folder
is broken
in
production
, and you
get to find
it while
you are
sitting in
front of it.

CHE
CK

Ru
n
np
m
run
bui
ld
on
yo
ur

ow
n
pro
jec
t
an
d
let
it
fini
sh.

Ho
w
lon
g
did
it
tak
e?

W
hat
eve
r
tha
t
nu
mb
er
is,
it
ha
pp
ens
aga
in
on
eve
ry

de
plo
y.

W
hat
is
insi
de
the
fol
der
?
Go
in
wit
h
cd
dis
t
or
cd
.ne
xt
an
d
run
ls -
F,
the
n
fin
d
the
HT
M
L
yo
ur
sou
rce

ne
ver
con
tai
ne
d.

Th
en
ser
ve
it
wit
h
pyt
ho
n3
-m
htt
p.s
erv
er
800
0
an
d
use
yo
ur
ow
n
site
co
mi
ng
out
of
a
file
ser
ver

tha
t
kn
ow
s
not
hin
g
ab
out
yo
ur
fra
me
wo
rk.
If
so
me
thi
ng
wo
rks
un
der
np
m
run
de
v
an
d
not
in
the
re,
yo
u
ha
ve

fou
nd
a
rea
l
bu
g
mo
nth
s
bef
ore
a
str
an
ger
wo
uld
ha
ve
fou
nd
it
for
yo
u.

☐ my
project
passes
this
check

Chapter 5
goes to
the shop
front and
the till.
What a
backend

is, and
why some
things can
never live
in the files
you just
looked at
however
well you
hide them.

why
does
the
till
stan
d in
a
differ
ent
roo
m?

Everythi
ng
delivere
d to the
browser
belongs
to
whoever
is
standing
there.

Everythin
g so far
has
happened
in one
room.
Files
arrive and
the
browser
draws

them. You
have now
read your
own
project
the way a
visitor
reads it. A
shop has a
second
room
behind the
counter
where the
till and
the keys
are, and
nobody
has to be
told why.
In a
project
both
rooms
look like
folders
sitting
next to
each other
in the
same
editor, so
the till
ends up
out on the
shop floor.

Which
room is

the
browser?

The
browser is
the front
one and
everything
in it
belongs to
whoever is
standing
there.

Every
file that
arrived is
listed in
the
Network
panel and
you can
click any
of them
and read
it,
including
the scripts
you wrote.
That will
not be
patched
one day,
because it
is what
delivery
means.

'My
code is
minified,
so nobody
can read
it,' you
may be
thinking.
Open one
of your

own
scripts in
that panel
and find
the button
marked
with
braces at
the
bottom of
the view.
The one
line
becomes
readable
code
again.
Squashing
a script
makes the
file smaller
and it
hides
nothing.

What is a
backend?

A backend
is a
program
on another
machine
that
answers
requests.
It runs
somewhere
else and
listens at a
door. You
talk to it
by sending
exactly

the kind of
request
you
watched
arrive in
chapter 1.

The
customer
orders and
the
kitchen
cooks. The
customer
does not
come into
the
kitchen,
because
the
kitchen is
a different
room. A
script
running in
the
browser is
a locked
cupboard
standing
in the
dining
room, and
the
customer
has all
evening.

What is
an API?

An API is
the menu.
A menu
tells you

what you
may order
and in
what
form. An
API tells
you which
requests
the
kitchen
accepts,
meaning
this
address
with this
method
and these
fields.

Anything
not on the
menu is
refused.

Writing
one is the
act of
deciding
what
strangers
may ask
you for.

The
verbs on
that menu
are the
methods.

GET
fetches
something,
POST
sends
something
new, PUT
replaces,
DELETE
removes.

A GET is

understood
everywhere to only
fetch and search
crawlers
act on
that
understanding. That
is how a
link which
deletes a
row gets
triggered
by a
crawler
that was
only
looking
around.

What do
the
numbers
mean?

Every
answer
comes
back with
a status
number
and MDN
carries the
definitions
if you
want them
properly.

200
means it
worked.
404 means

the
kitchen
understood
you
perfectly
and has no
such dish.
500 means
the
kitchen
broke
while
cooking,
so the
fault is on
their side.

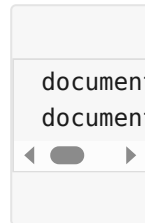
The
two that
get
confused
constantly
are 401
and 403.
A 401
means the
kitchen
does not
know who
you are,
while a
403 means
it knows
exactly
who you
are and
refuses
anyway.
Identity
and
permission
are
separate
systems
with
separate
fixes, and

telling
them
apart is
the whole
of chapter
26.

Can you
break
your own
rule?

Find
something
in your
own
project
that the
page
refuses to
let you do.
A submit
button
that stays
disabled
until the
form is
valid, a
field with
a
maximum
length, a
section
that only
appears
when you
are logged
in.
Anything
the page
decides.
Open
the
developer

tools and go to the Console tab. Then type one of these, matching whichever rule you found.



The button is live. The field takes as much as you want. You did not find a hole in your code, you retyped a line of it, and every visitor has the same console you just used.

That is why a check written into your page is a suggestion. Your script runs on their machine. A check

that holds
has to run
in a room
they
cannot
reach,
which is
the room
this
chapter is
about.

Where
does the
key go?

Into the
kitchen
and never
into a
request
the
customer
can read.

stitchu
sends a
photograph
to a
vision
model and
that
model
charges
per call.
The key
that pays
for it
belongs in
the till, so
the
browser
never
holds it.
The

photograph
h goes to
a
Cloudflare
Worker,
the
Worker
holds the
key and
calls the
model,
and the
answer
comes
back.
Anyone
can open
the
Network
panel on
stitchu
and read
every
request
that leaves
their
machine
and find
nothing in
them to
take.

The
wrong
version is
one line of
difference.
The
browser
calls the
model
directly
with the
key
attached
to the
request. It

works on
the first
try (which
is the
trouble)
because
from
behind the
counter
both
versions
look the
same to
you. Every
visitor
now holds
a working
key billed
to your
account,
and
chapter 31
is about
the size of
that bill.

CHE
CK

Op
en
yo
ur
ow
n
pro
jec
t
wit
h
the
Ne
tw

ork
pa
nel
op
en
an
d
use
it
unt
il
req
ues
ts
sta
rt
ap
pe
ari
ng.

W
her
e is
eac
h
req
ues
t
goi
ng
?
Re
ad
the
ad
dre
ss
of
eac
h

one
. If
it
goe
s
str
aig
ht
to
so
me
bo
dy
els
e's
ser
vic
e
the
n
yo
ur
bro
ws
er
is
tal
kin
g
to
tha
t
ser
vic
e
dir
ect
ly
an
d

every-
thing
hiding
in
that
request
is
public.

What
in
your
code
does
cost
money
or
grants
access
?

An
API
I
key
, a
database
URL

L,
an
ad
mi
n
tok
en.
W
her
e
do
es
it
live
rig
ht
no
w?

If
it
rea
che
d
the
bro
ws
er
the
n
it
is
pu
bli
c,
an
d
the
ne
xt
tw

o
cha
pte
rs
are
ab
out
mo
vin
g
it.

☐ my
project
passes
this
check

Chapter 6
goes to
where
data lives,
and why a
file works
perfectly
until two
people use
your
project at
the same
time.